

Arhitectura calculatoarelor

Curs 9

9. Planificarea dinamică a execuției

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

Obiective

1. Clasificarea hazardurilor de date
2. Pipeline avansat
3. Exploatarea ILP în bucle
4. Planificarea dinamică: Tomasulo

9.1 Clasificarea hazardurilor de date

9.1.1 Read After Write (RAW) – dependență reală, întâlnit până acum la MIPS pipeline

Instr₁: R3 ← R2 + R1

Instr₂: R5 ← R3 + R4 – va citi valoarea R3 înainte să fie scrisă de Instr₁ => **hazard RAW**

- **Problema:** Se citește o valoare înainte să fie actualizată.
- Este cel mai frecvent hazard de date.
- **Soluție:** Înaintare (Forward) și așteptare (Stall).

9.1.2 Write After Write (WAW) – dependență de nume, apare la pipeline avansat

Instr₁: R3 ← R2 x R1 – înmulțirea durează mai multe cicluri și va scrie R3 după Instr₂ => **hazard WAW**

Instr₂: R3 ← R4 + R5

- **Problema:** Nu se respectă ordinea scrierii. După execuția instrucțiunilor R3 va avea rezultatul înmulțirii și nu al adunării.
- Apare la pipeline care permite instrucțiuni concomitente cu număr diferit de cicluri de execuție – nu e cazul la MIPS pipeline cu 5 etaje.

9.1 Clasificarea hazardurilor de date

9.1.3 Write After Read (WAR) – antidependență, apare la pipeline avansat

Instr₁: R1 ← R9 x R7

Instr₂: R4 ← R2 + R1 – așteaptă să fie disponibil R1 de la Instr₁ (Stall)

Instr₃: R2 ← R5 - R6 – se execută mai repede și va scrie valoarea R2 înainte să fie citită de Instr₂ => Instr₂ va folosi rezultatul R2 de la Instr₃ => **hazard WAR**

- **Problema:** Se scrie o valoare înainte să fie citită de o instrucțiune anterioară.
- Apare la pipeline care permite execuția unei instrucțiuni chiar dacă altele anterioare sunt în așteptare (Stall) – nu e cazul la MIPS pipeline cu 5 etaje.

9.2 Pipeline avansat

Se extinde MIPS cu operații complexe, astfel:

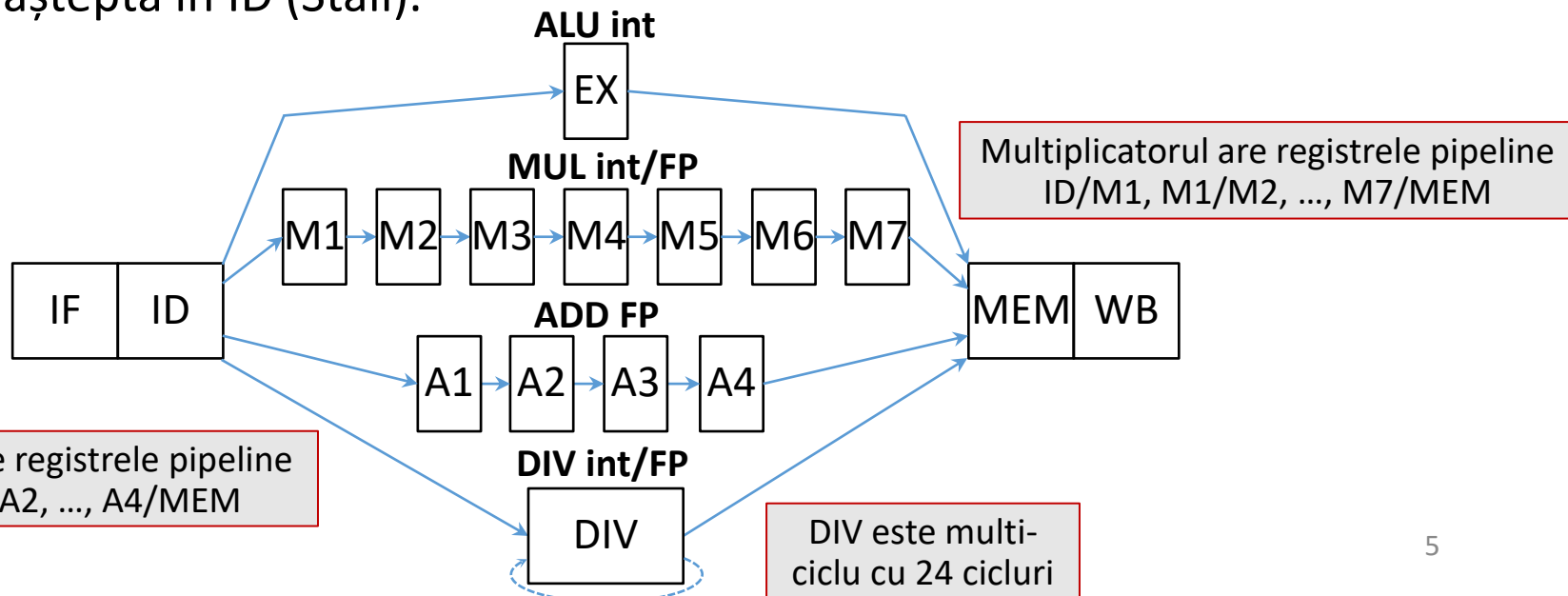
- înmulțire/împărțire cu numere întregi;
- operații cu numere reale în virgulă mobilă (FP – floating point).

Obs: 1. Sunt amplasate în etajul EX unități funcționale multi-ciclu (ex. DIV) sau pipeline de lungime variabilă. **Notă:** DIV pipeline ar fi prea costisitor.

2. În etajul ID se lansează în execuție 1 instrucțiune/ciclu.

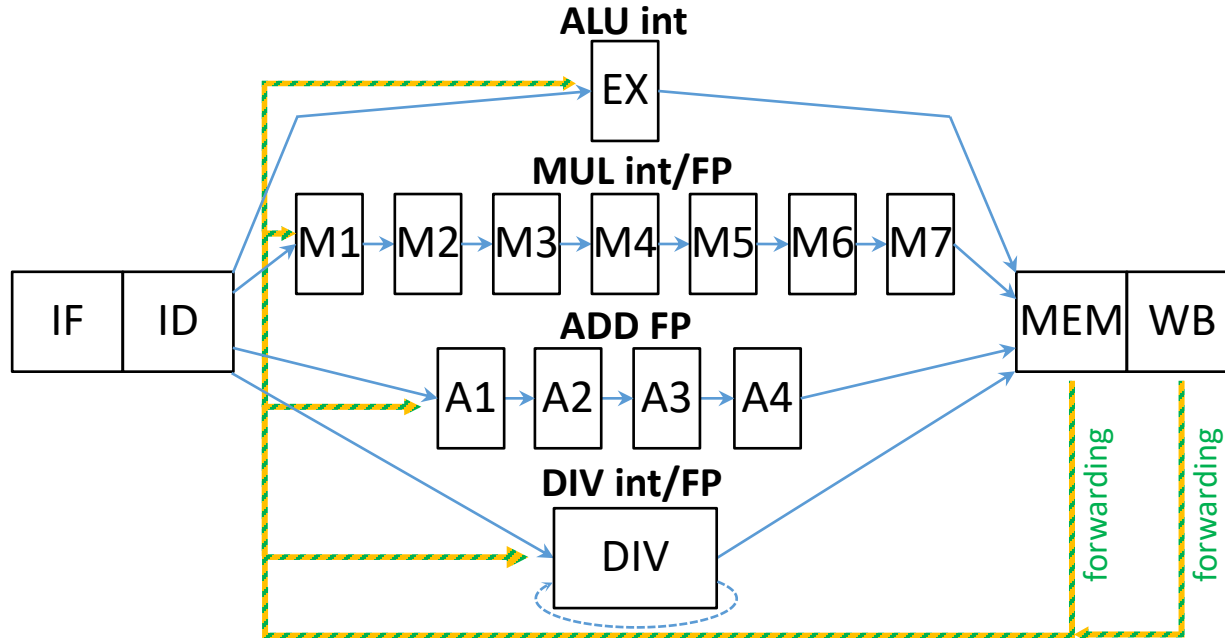
3. În etajul EX se pot executa operații, în paralel, pe unitățile funcționale.

4. DIV poate executa o operație. Cât timp este ocupat, împărțirile ulterioare vor aștepta în ID (Stall).



9.2 Pipeline avansat

- Se face **înaintare** din MEM și WB în EX, M1, A1 și DIV folosind MUX-uri.



- Obs:** La operații load se poate înainta numai din WB.

Proprietăți ale operațiilor

- *Latența:** cicluri de Stall necesare dacă o instrucțiune folosește rezultatul celei de dinainte = nr. de etaje – 1 datorită înaintării * definiția aceasta e valabilă doar în cadrul acestui curs.
- Interval de inițiere/repetare:** cicluri necesare între operații de același tip.

Operație	Latență	Interval de inițiere
Aritmetică-logică pe întregi	0	1
Load memorie (întregi/FP)	1	1
Adunare FP	3	1
Înmulțire întregi/FP	6	1
Împărțire întregi/FP	23	24

9.2 Pipeline avansat

Diagrama execuției pipeline a instrucțiunilor

Legendă

roșu ⇔ datele sunt necesare

albastru ⇔ datele sunt produse

Instrucțiune

MULD	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADDD	IF	ID	A1	A2	A3	A4	MEM	WB			
LD/LW	IF	ID	EX	MEM	WB						
SD/SW	IF	ID	EX	MEM	WB						

Exemplu de execuție cu hazarduri RAW rezolvate prin forwarding și stall

Instrucțiune	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉	C ₁₀	C ₁₁	C ₁₂	C ₁₃	C ₁₄	C ₁₅	C ₁₆	C ₁₇	C ₁₈
LD F4, 0(\$t2)	IF	ID	EX	MEM	WB													
MULD F0, F4, F6		IF	ID	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB					
ADDD F2, F0, F8			IF	IF	ID	ID	ID	ID	ID	ID	ID	A1	A2	A3	A4	MEM	WB	
SD F2, 0(\$t2)					IF	IF	IF	IF	IF	IF	IF	ID	EX	EX	EX	EX	MEM	WB

Excepție: Instrucțiunea SD începe etajul EX imediat după ID deoarece \$t2 este disponibil pentru calculul adresei. În ciclul C₁₆ face **stall** în loc de **fw** pentru a evita suprapunerea cu ADDD în MEM.

9.2 Pipeline avansat

Exemplu de **hazard structural** la blocul de registre

Instrucțiune	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉	C ₁₀	C ₁₁
MULD F2 , F1, F0	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADD \$t1, \$t1, \$t0		IF	ID	EX	MEM	WB					
ADD \$t3, \$t2, \$t0			IF	ID	EX	MEM	WB				
ADDD F2 , F3, F4				IF	ID	A1	A2	A3	A4	MEM	WB

Scrierea simultană în ciclul C₁₁ este hazard structural. Se poate rezolva prin întârzierea lui ADDD în etajul ID pentru o perioadă de ceas.

Exemplu de **hazard WAW**

Instrucțiune	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉	C ₁₀	C ₁₁
MULD F2 , F1, F0	IF	ID	M1	M2	M3	M4	M5	M6	M7	MEM	WB
ADD \$t1, \$t1, \$t0		IF	ID	EX	MEM	WB					
ADDD F2 , F3, F4			IF	ID	A1	A2	A3	A4	MEM	WB	

Scrierea lui ADDD înainte de MULD este hazard WAW. Se poate rezolva prin întârzierea lui ADDD în etajul ID până la finalizarea lui MULD.

9.2 Pipeline avansat

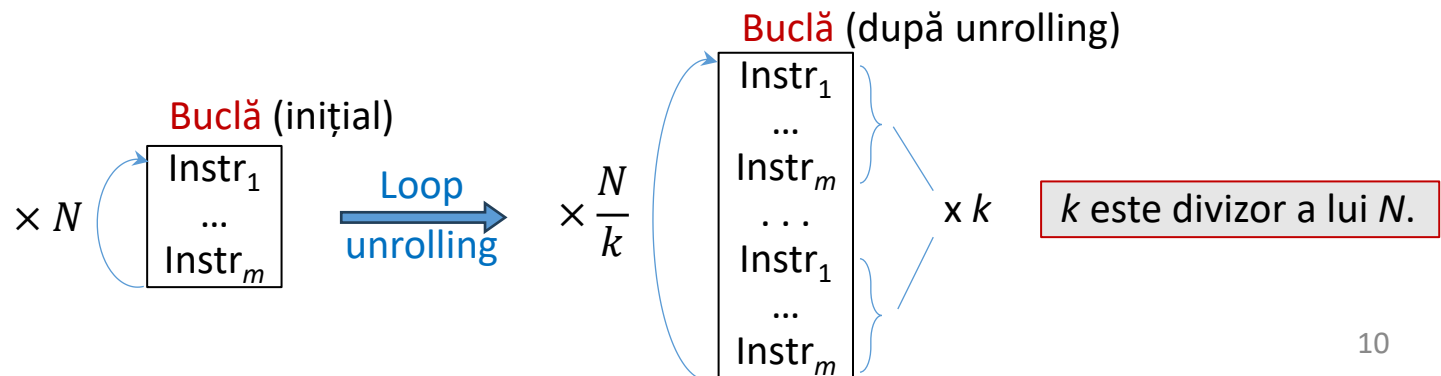
Probleme întâmpinate

- Divizor multi-ciclu => apare **hazard structural** la împărțiri succesive.
 - **Soluție:** se așteaptă (**stall**) în etajul ID până la eliberarea divizorului.
- Pipeline cu multe etaje => crește frecvența **hazardurilor RAW**.
 - **Soluție:** **stall** în etajul ID până când operanzii sunt disponibili.
- Număr diferit de etaje în EX => instrucțiunile nu ajung în ordine în etajul WB:
 1. Poate apărea **hazard structural** (scriere simultană) la blocul de registre.
 - **Soluție:** **stall** în etajul ID.
 2. Poate apărea **hazard WAW**.
 - **Soluție:** **stall** în etajul ID.
- **Obs:** **Hazard WAR** nu poate apărea deoarece se face stall în etajul ID, dacă operanzii nu sunt disponibili, blocând instrucțiunile următoare.
- **Concluzie:** Este necesară reducerea hazardurilor prin soluții software și hardware.

9.3 Exploatarea paralelismului la nivel de instrucțiune (ILP) în bucle

Planificarea instrucțiunilor (scheduling) și despachetarea buclelor (loop unrolling)

- Sunt tehnici de reducere a hazardurilor la nivel software (**de către compilator**).
- Planificarea instrucțiunilor** (instruction scheduling) = reordonarea instrucțiunilor pentru evitarea hazardurilor.
 - Separă instrucțiunile dependente prin intercalare cu instrucțiuni independente.
 - Latența* instrucțiunilor determină câte instrucțiuni trebuie să separe instrucțiunile dependente pentru a evita hazardul (**stalls**).
- Despachetarea buclelor** (loop unrolling) = repetarea secvențială a buclei în cadrul unei noi bucle mai mari, **folosind registre suplimentare la fiecare repetare**. Avantaje:
 - Reduce numărul de instrucțiuni pentru controlul buclei (operații contor/salturi).
 - Crește numărul de instrucțiuni din corpul buclei => mai multe instrucțiuni la dispoziție pentru o planificare eficientă.



9.3 Exploatarea paralelismului la nivel de instrucțiune (ILP) în bucle

Planificarea instrucțiunilor (scheduling) și despachetarea buclelor (loop unrolling) (cont)

▪ Detalii de arhitectură:

- Pentru simplificare, presupunem că toate unitățile pipeline pentru operații FP au 4 etaje: EX_1 , EX_2 , EX_3 , EX_4 .
- Salturile branch se realizează în etajul ID și nu se face *Flush* => este necesar 1 NoOp sau *branch delay slot*.

Notă: Compilatorul cunoaște întârzierile apărute (*stalls*) la hazardurile RAW:

Hazard RAW – dependență de date		
Instrucțiunea care produce rezultatul	Instrucțiunea care folosește rezultatul	Latența (nr. de <i>stalls</i>)
FP	FP	3
FP	store	2
load	FP	1
load	store	0
ALU Int	ALU Int	0
ALU Int	branch	1
load	branch	2

9.3 Exploatarea paralelismului la nivel de instrucțiune (ILP) în bucle

Planificarea instrucțiunilor (scheduling) și despachetarea buclelor (loop unrolling) (cont)

Exemplu: Adunarea valorii double S la un șir de numere reale în dublă precizie.

C++:

```
for (i=1000; i>0; i--)  
    x[i] += S;
```

Asamblare MIPS:

```
Loop: LD    F0, 0($t1)    ; încarcă element  
      ADDD  F4, F0, F2    ; adună S stocat în F2  
      SD    F4, 0($t1)    ; stochează element  
      SUBI  $t1, $t1, 8    ; actualizează contor  
      BNEZ  $t1, Loop     ; salt la start buclă  
      NoOp                ; rezolvare hazard control
```

Execuție în pipeline => **10** cicluri / iterație

Execuție cu scheduling => **6** cicluri / iterație

Ciclu

```
Loop: 01: LD    F0, 0($t1)  
      02: Stall  
      03: ADDD  F4, F0, F2  
      04: Stall  
      05: Stall  
      06: SD    F4, 0($t1)  
      07: SUBI  $t1, $t1, 8  
      08: Stall  
      09: BNEZ  $t1, Loop  
      10: NoOp
```

Ciclu

```
Loop: 01: LD    F0, 0($t1)  
      02: SUBI  $t1, $t1, 8  
      03: ADDD  F4, F0, F2  
      04: Stall  
      05: BNEZ  $t1, Loop  
      06: SD    F4, 8($t1)
```



Se reordonează instrucțiunile.
Obs: SD devine *branch delay slot* și nu mai e nevoie de NoOp.

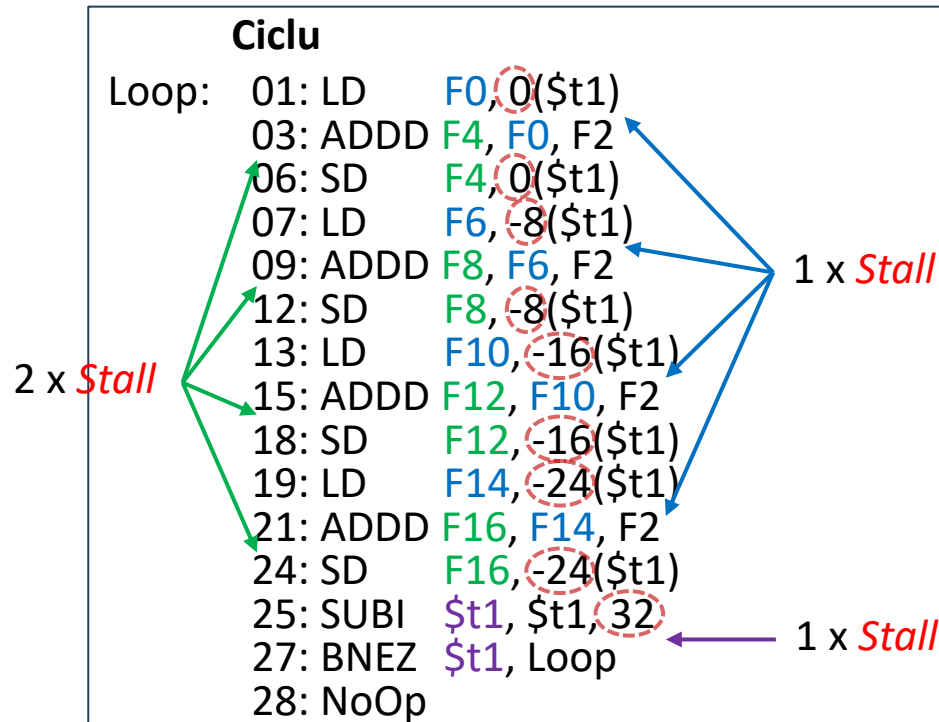
Se adună 8 pentru a compensa efectul SUBI mutat deasupra.

9.3 Exploatarea paralelismului la nivel de instrucțiune (ILP) în bucle

Planificarea instrucțiunilor (scheduling) și despachetarea buclelor (loop unrolling)

Despachetarea buclei fără scheduling => 28 cicluri / 4 iterații = **7** cicluri / iterație

(cont)



Important: Fiecare iterație mică din bucla mare **trebuie să folosească noi seturi de registre**, pentru a genera independențe între instrucțiuni, facilitând astfel posibilitatea unei reordonări eficiente.

Observații:

- Fiecare 4 iterații consecutive inițiale formează o nouă iterație extinsă la un număr mai mare de instrucțiuni. Se consideră lungimea șirului multiplu de 4.
- Se utilizează suplimentar 6 registre: F6, F8, F10, F12, F14, F16, câte 2 pentru fiecare nouă iterație. Folosind seturi de registre diferite, **se introduc independențe între operanzi din iterații diferite**, care pot fi folosite în procesul de reordonare (scheduling).
- Elementele de control ale buclei (scăderea contorului și branch-ul) apar o singură dată în noua buclă. Contorul este actualizat cât pentru 4 iterații originale.
- Se folosesc offset-uri recalculate la adresele de memorie (0, -8, -16, -24) pentru acces la adresele corecte.
- **Dezavantaj:** se păstrează hazardurile => **Stalls**.

9.3 Exploatarea paralelismului la nivel de instrucțiune (ILP) în bucle

Planificarea instrucțiunilor (scheduling) și despachetarea buclelor (loop unrolling) (cont)

Despachetarea buclei + scheduling => 14 cicluri / 4 iterații = **3.5** cicluri / iterație

Ciclu

Loop: 01: LD F0, 0(\$t1)
02: LD F6, -8(\$t1)
03: LD F10, -16(\$t1)
04: LD F14, -24(\$t1)
05: ADDD F4, F0, F2
06: ADDD F8, F6, F2
07: ADDD F12, F10, F2
08: ADDD F16, F14, F2
09: SD F4, 0(\$t1)
10: SD F8, -8(\$t1)
11: SUBI \$t1, \$t1, 32
12: SD F12, 16(\$t1)
13: BNEZ \$t1, Loop
14: SD F16, 8(\$t1)

Observații:

- Se grupează citirile din memorie, apoi adunările și apoi stocările în memorie. Practic, reordonarea intercalează instrucțiuni independente, provenite de la iterații diferite, **care folosesc seturi de registre diferite**. După BNEZ se plasează 1 *branch delay slot* pt. a elimina NoOp-ul.
- După reordonare, se evidențiază avantajul folosirii registrelor suplimentare pentru eliminarea hazardurilor => dispar **Stalls** și NoOps.
- Limite:
 1. Crește dimensiunea codului => poate îngreuna accesul la memoria de instrucțiuni provocând întâzieri mai frecvente.
 2. Crește numărul de registre utilizate => rămân mai puține registre libere pentru alte scopuri.

Important: Fiecare iterație mică din bucla mare **trebuie să folosească noi seturi de registre**, pentru a genera independențe între instrucțiuni, facilitând astfel posibilitatea unei reordonări eficiente.

Se adună 16, respectiv 8, pentru a compensa efectul instrucțiunii SUBI mutată deasupra.



9.4 Planificarea dinamică: Tomasulo

Robert Tomasulo → tehnică **hardware** de exploatare a paralelismului instrucțiunilor.

9.4.1 Motivație

- Compilatorul nu poate depista toate dependențele.
- Dacă o parte din atribuții sunt rezolvate hardware, compilatoarele pot fi mai simple.
- **Dezavantajul execuției *in-order***: o instrucțiune blocată (Stall) în etajul ID, blochează lansarea în execuție a tuturor instrucțiunilor următoare.

Ex: DIVD **F3**, F2, F5

 ADDD F10, F0, **F3** ; *blocat în ID din cauza hazardului RAW pe F3. Blochează instr. următoare*

 SUBD F5, F6, F8 ; *blocat în IF datorită ADDD, deși s-ar putea executa pe o unitate liberă*

9.4.2 Obiective

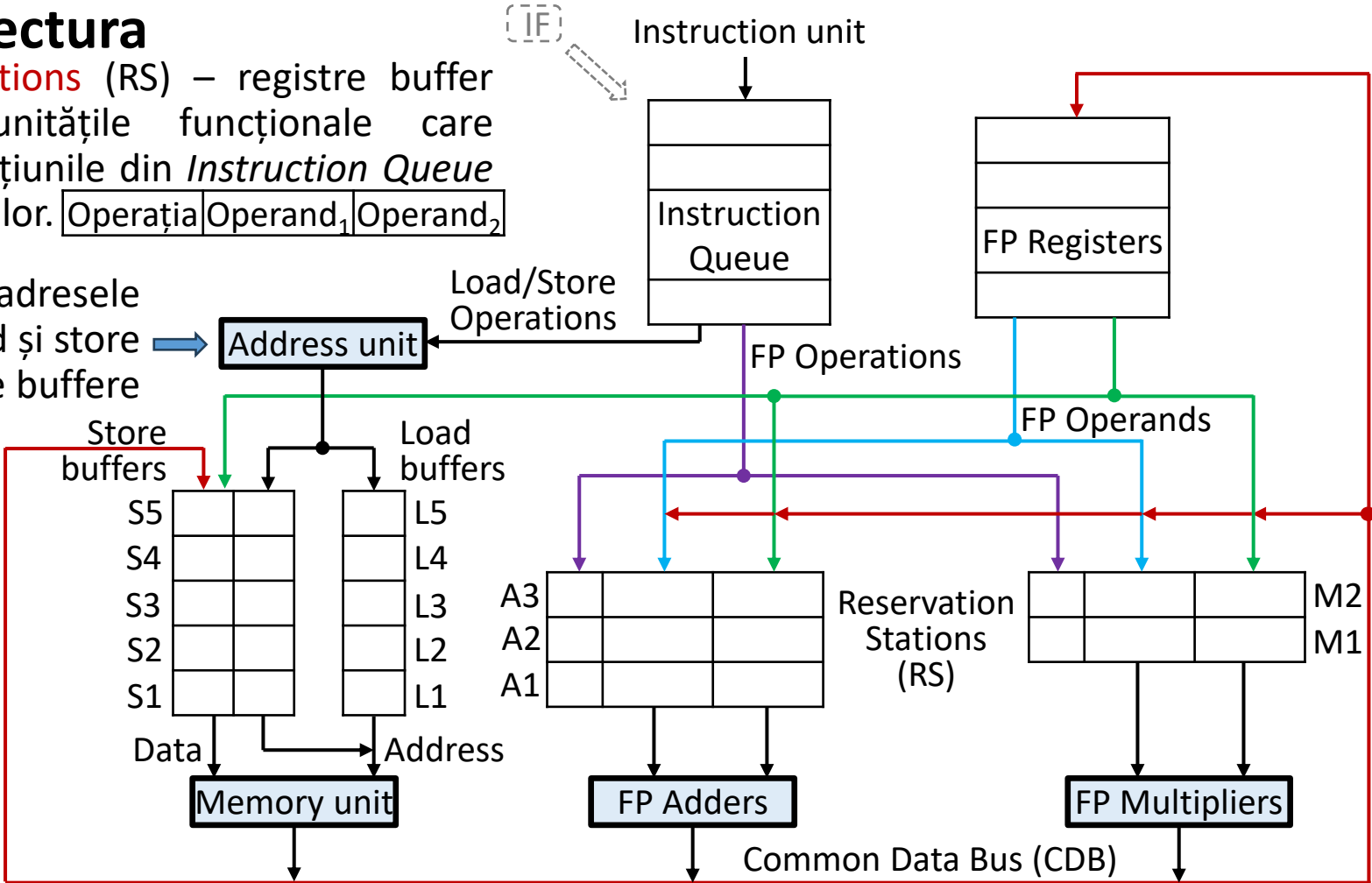
- **Planificarea dinamică** a execuției pentru a ușura sarcina compilatoarelor = execuția și finalizarea *out-of-order* (*out-of-order execution and completion*), adică să se poată executa noi instrucțiuni, chiar dacă instrucțiunile anterioare sunt blocate în așteptarea operanzilor => finalizarea nu va respecta tot timpul ordinea inițială.
- **Rezolvarea hazardurilor RAW**, prin folosirea de buffere de stocare (*reservation stations*) cu rol de registre noi. **Rezolvarea hazardurilor WAR și WAW** fără așteptări (Stalls), prin redenumirea registrelor (*register renaming*).

9.4.3. Istoric

- Dezvoltat inițial pentru IBM 360/91 (1966). Relansat ulterior în anii '90 pe Alpha 21264 (1998), MIPS R10000 (1996), Pentium II (1997), PowerPC 604 (1994).

9.4.4 Arhitectura

Reservation stations (RS) – registre buffer asociate la unitățile funcționale care “preiau” instrucțiunile din *Instruction Queue* pentru execuția lor.



Notă: Instrucțiunile ajung în *Instruction Queue* în ordinea de execuție, conform programului.

FP Adders – realizează adunare/scădere cu numere reale (FP = floating-point).

FP Multipliers – realizează înmulțire/împărțire cu numere reale (FP = floating-point).

Address unit – calculează adresele de acces load/store la memorie în ordinea de execuție.

Store buffer – stochează adresa de scriere și data de scris în memorie.

Data	Adresa
------	--------

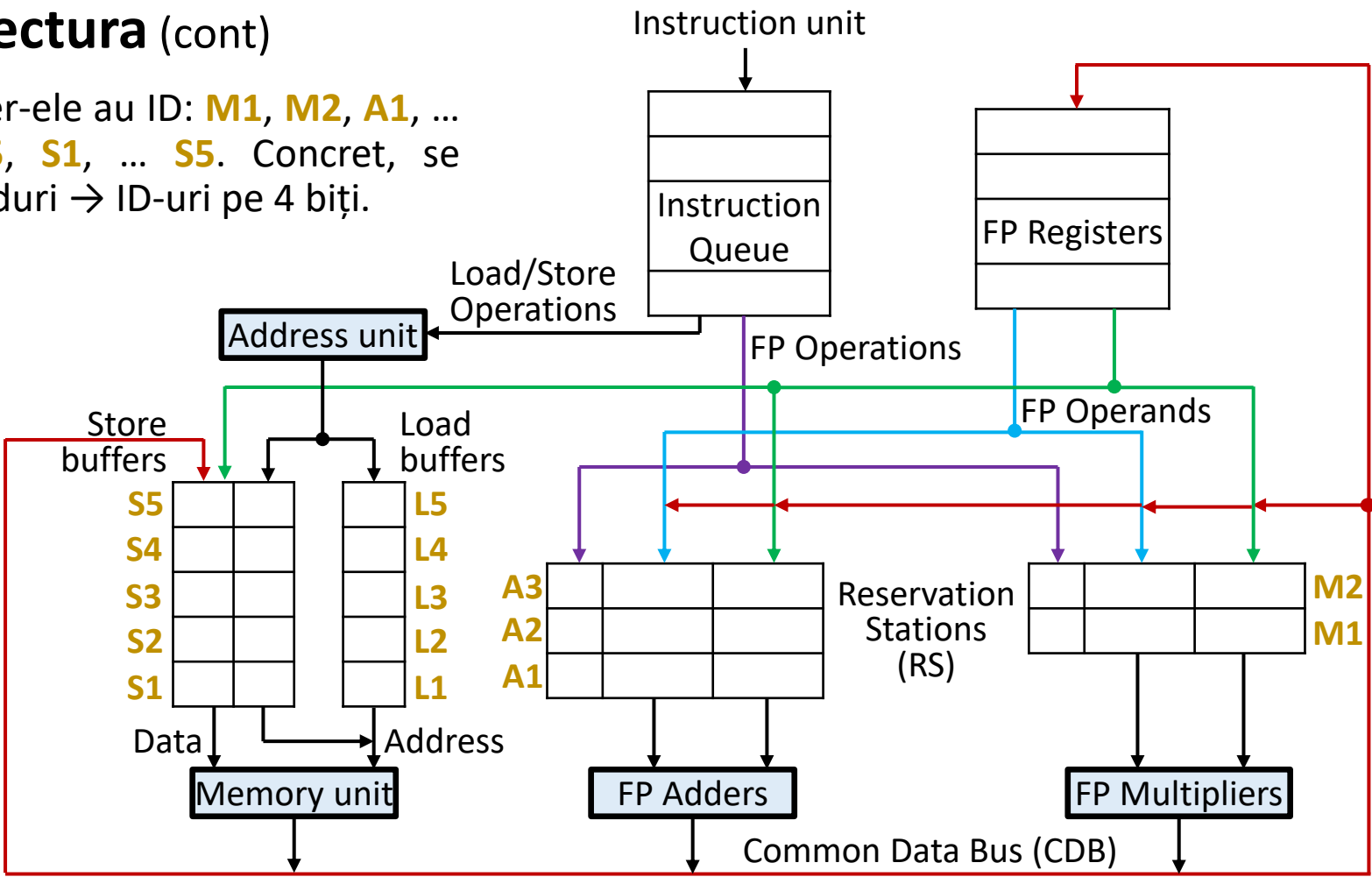
Load buffer – stochează adresa de citire din memorie.

Adresa

Obs: Buffer-ele Load/Store permit efectuarea accesului la memorie în paralel cu operații aritmetice-logice. Buffer-ele sunt tratate ca stații RS cu conținut diferit.

9.4.4 Arhitectura (cont)

Stațiile și buffer-ele au ID: **M1, M2, A1, ... A3, L1, ... L5, S1, ... S5**. Concret, se folosesc 15 coduri → ID-uri pe 4 biți.



Redenumirea registrelor (register rename): La lansare, instrucțiunilor care scriu un registru li se alocă o stație RS sau un load buffer. La alocare, registrul destinație primește temporar ID-ul stației/buffer-ului, până valoarea nou calculată devine disponibilă pe CDB. În acest interval, instrucțiunile nou lansate, care citesc registrul, folosesc ID-ul temporar ca referință la registru.

Common Data Bus (CDB) – magistrală prin care se emite simultan rezultatul (broadcast), de la o unitate funcțională sau memorie, către blocul de registre, stațiile RS și buffer-ele store.

- CDB are 64 biți date + 4 biți ID-ul stației RS sau al Load buffer-ului pentru care se difuzează rezultatul. Valoarea difuzată va înlocui toate referințele la ID, unde este cazul.

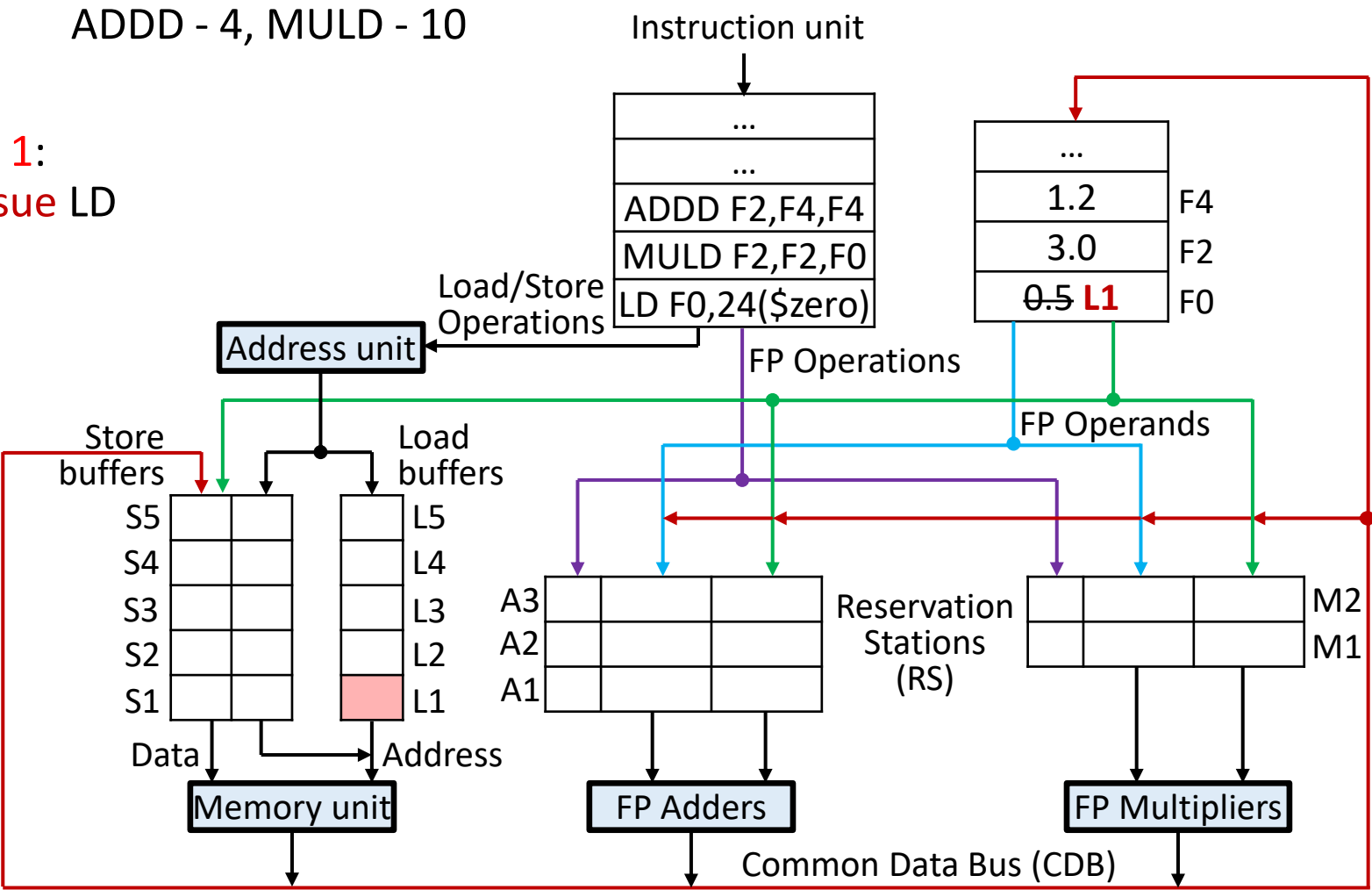
9.4 Planificarea dinamică: Tomasulo

9.4.5 Fazele de execuție a unei instrucțiuni

1. **Issue** – Lansarea instrucțiunii din vârful cozii de instrucțiuni (*Instruction Queue*).
 - La operație FP se alocă o stație RS disponibilă și se trimit valorile operanzilor către ea, dacă se găsesc în registre, altfel se înlocuiesc cu ID-ul stațiilor care îi calculează.
 - La operație load sau store se alocă un buffer disponibil. Câmpul *data* la un store buffer va fi valoarea registrului sau ID-ul stației care îl calculează.
 - Dacă nu sunt stații RS/buffer-e libere (**hazard structural**) se face **Stall**.
 - Redenumirea registrului (*register rename*): dacă instrucțiunea scrie un registru, acesta se redenumeste cu ID-ul stației/buffer-ului alocat, până la finalizare (**rezolvă hazardurile WAW și WAR**).
2. **Execute** – Execuția unei instrucțiuni după lansare.
 - Operanzii indisponibili sunt așteptați pe CDB pe baza ID-ului de broadcast.
 - Când toți operanzii sunt disponibili (**rezolvă hazardurile RAW**) și toate branch-urile anterioare sunt finalizate (**rezolvă hazardurile de control**), se execută operația. Dacă este nevoie de **Flush**, se eliberează stația/buffer-ul și se renunță la redenumire.
 - Un load are 2 pași (**nu cicluri de ceas!**) de execuție: calcul adresă + citirea din memorie.
 - Un store realizează doar calculul adresei (scrierea în memorie are loc în *write result*).
3. **Write result** – Scrierea rezultatului după execuție (înlocuiește *Write-Back*).
 - Rezultatul și ID-ul se difuzează pe CDB către stațiile, registrele și buffer-ele care îl așteaptă. Se poate difuza un rezultat pe ciclu.
 - Un store nu difuzează pe CDB, doar așteaptă valoarea și apoi o scrie în memorie.
 - Stația/buffer-ul care difuzează/scrie memoria se eliberează. Se renunță la redenumire.

Exemplu Timpi execuție [cicluri clk]: LD - 8 (1 calcul adresă + 7 citire memorie, ADDD - 4, MULD - 10

- Ciclu 1:
- Issue LD



Redenumirea registrelor

- Evenimente:
- Se alocă Load buffer L1 (marcat cu roșu) pentru citirea F0 din memorie.
 - F0 se redenumeste temporar la L1.



LD F0,24(\$zero) → LD L1,24(\$zero)

MULD F2,F2,F0

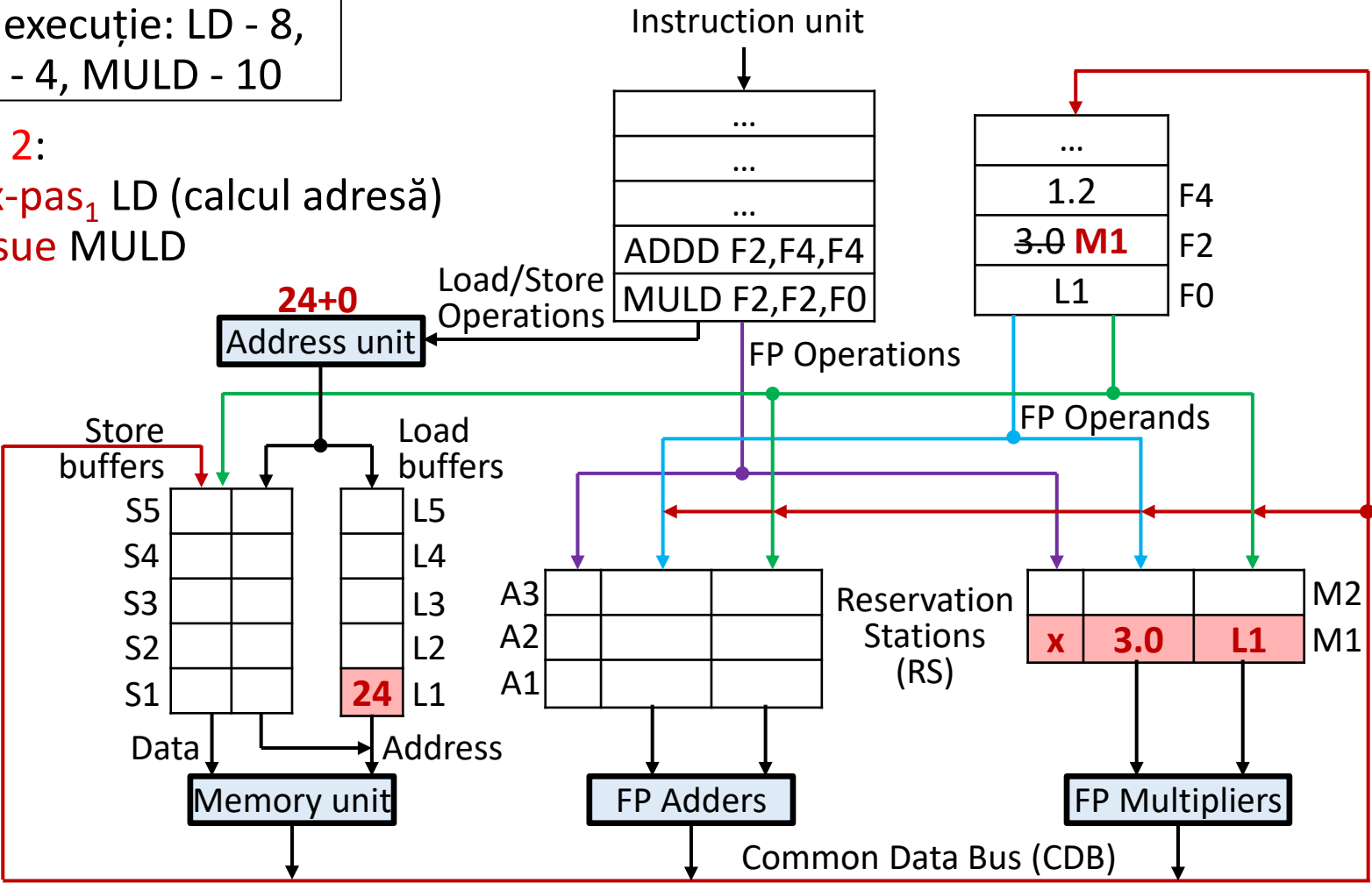
ADDD F2,F4,F4

Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 2:

- **Ex-pas₁** LD (calcul adresă)
- **Issue** MULD



Redenumirea registrelor

Evenimente:

- Se alocă stația M1 (marcat cu roșu) pentru MULD.
- F2 se redenumeste temporar la M1.



LD F0,24(\$zero)

MULD F2,F2,F0

ADDD F2,F4,F4

LD L1,24(\$zero)

MULD **M1**,3.0,**L1**

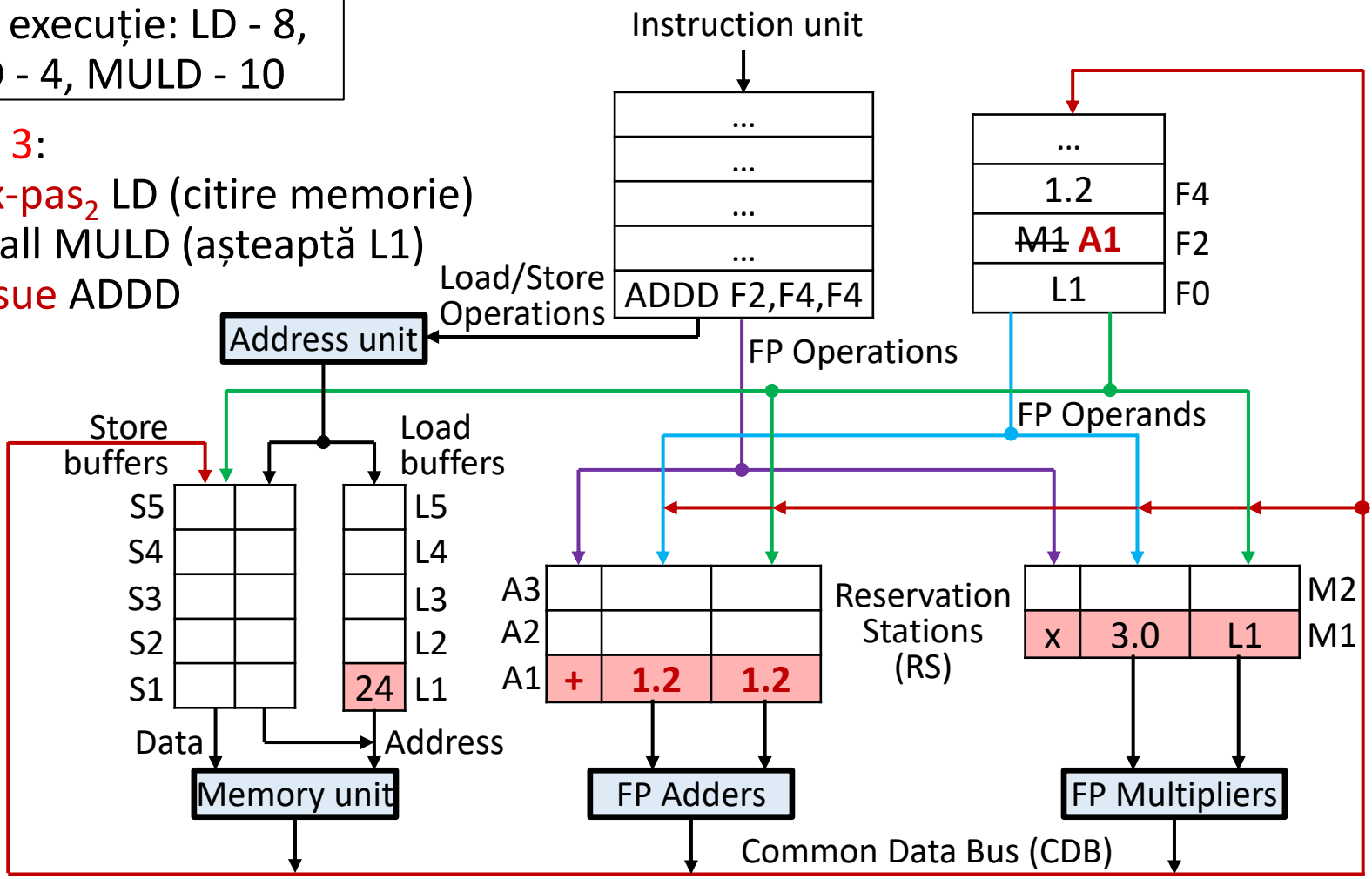
Obs: Stația M1 primește valoarea 3.0 de la reg. F2, apoi F2 este redenumit.

Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 3:

- **Ex-pas₂** LD (citire memorie)
- Stall MULD (așteaptă L1)
- **Issue** ADDD



Redenumirea registrelor

LD F0,24(\$zero)	LD L1,24(\$zero)
MULD F2,F2,F0	MULD M1,3.0,L1
ADDD F2,F4,F4	ADDD A1 ,1.2,1.2

Evenimente:

- Se alocă stația A1 (marcat cu roșu) pentru ADDD.
- F2 se redenumeste la A1 și **astfel, va primi numai rezultatul ultimei scrieri în el** (elimină hazardul WAW).

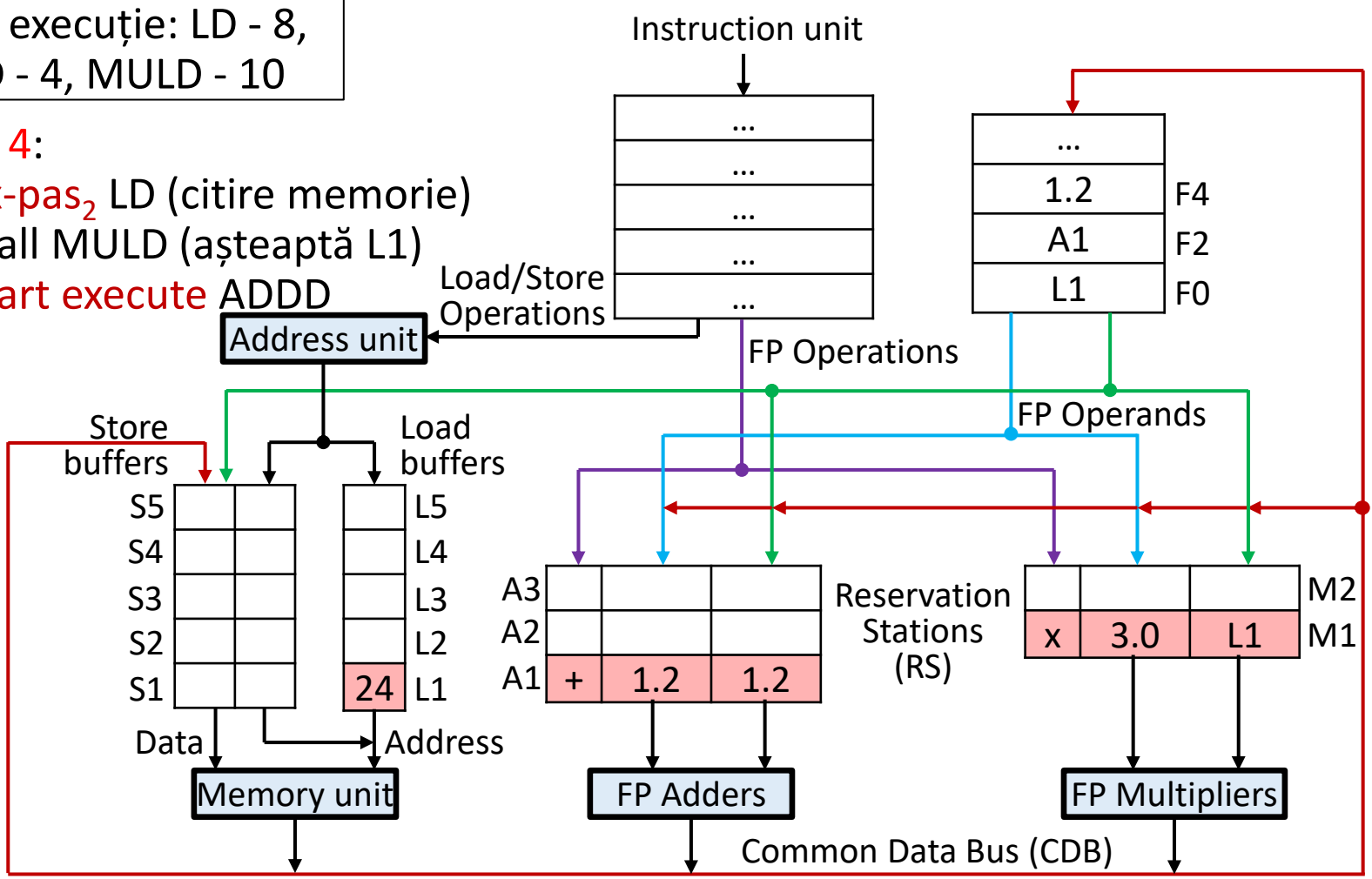


Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 4:

- **Ex-pas₂** LD (citire memorie)
- Stall MULD (așteaptă L1)
- **Start execute** ADDD



Evenimente:

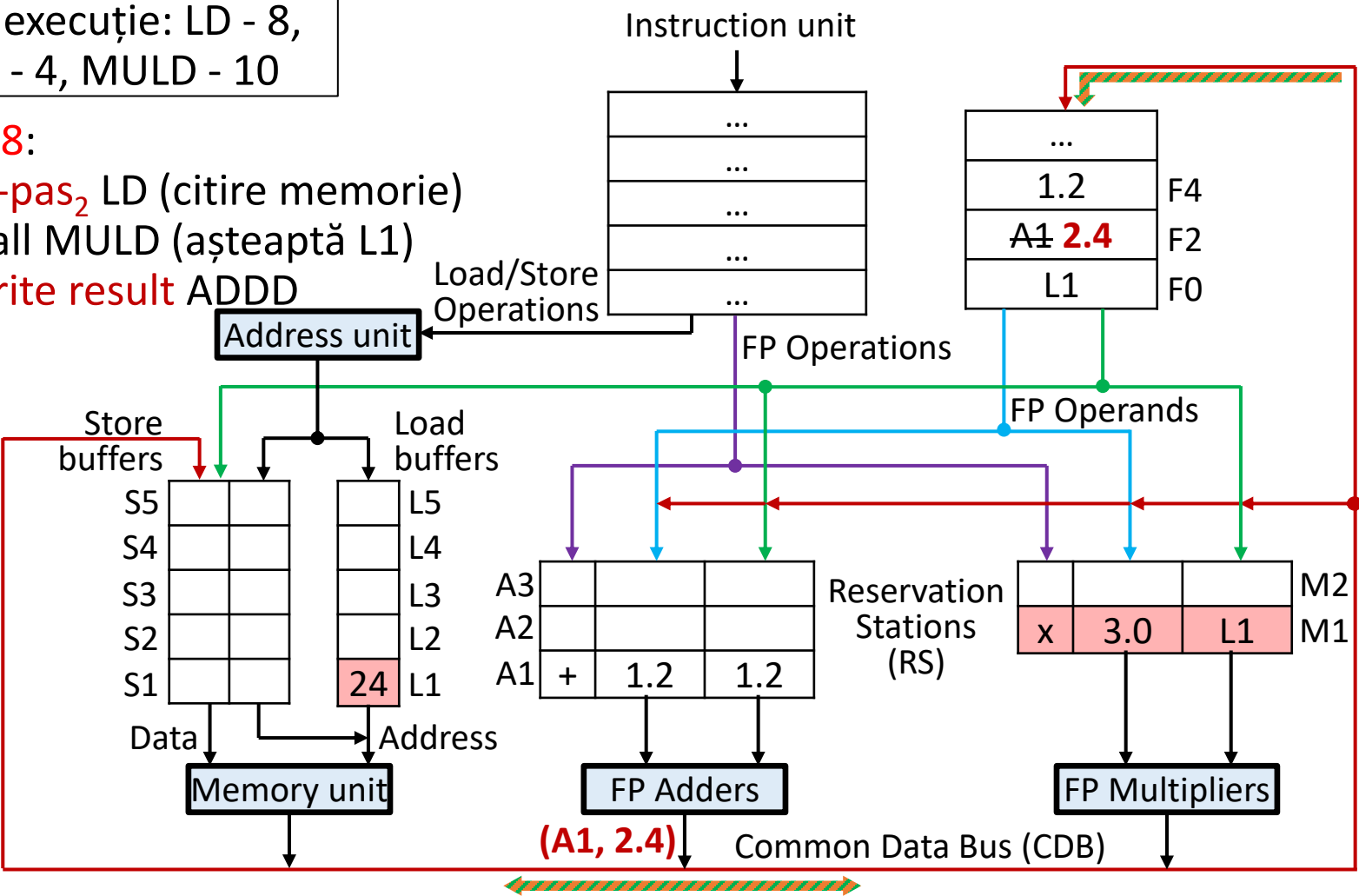
- Stația A1 are toți operanzii, începe execuția ADDD.

Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 8:

- **Ex-pas₂** LD (citire memorie)
- Stall MULD (așteaptă L1)
- **Write result** ADDD



Evenimente:

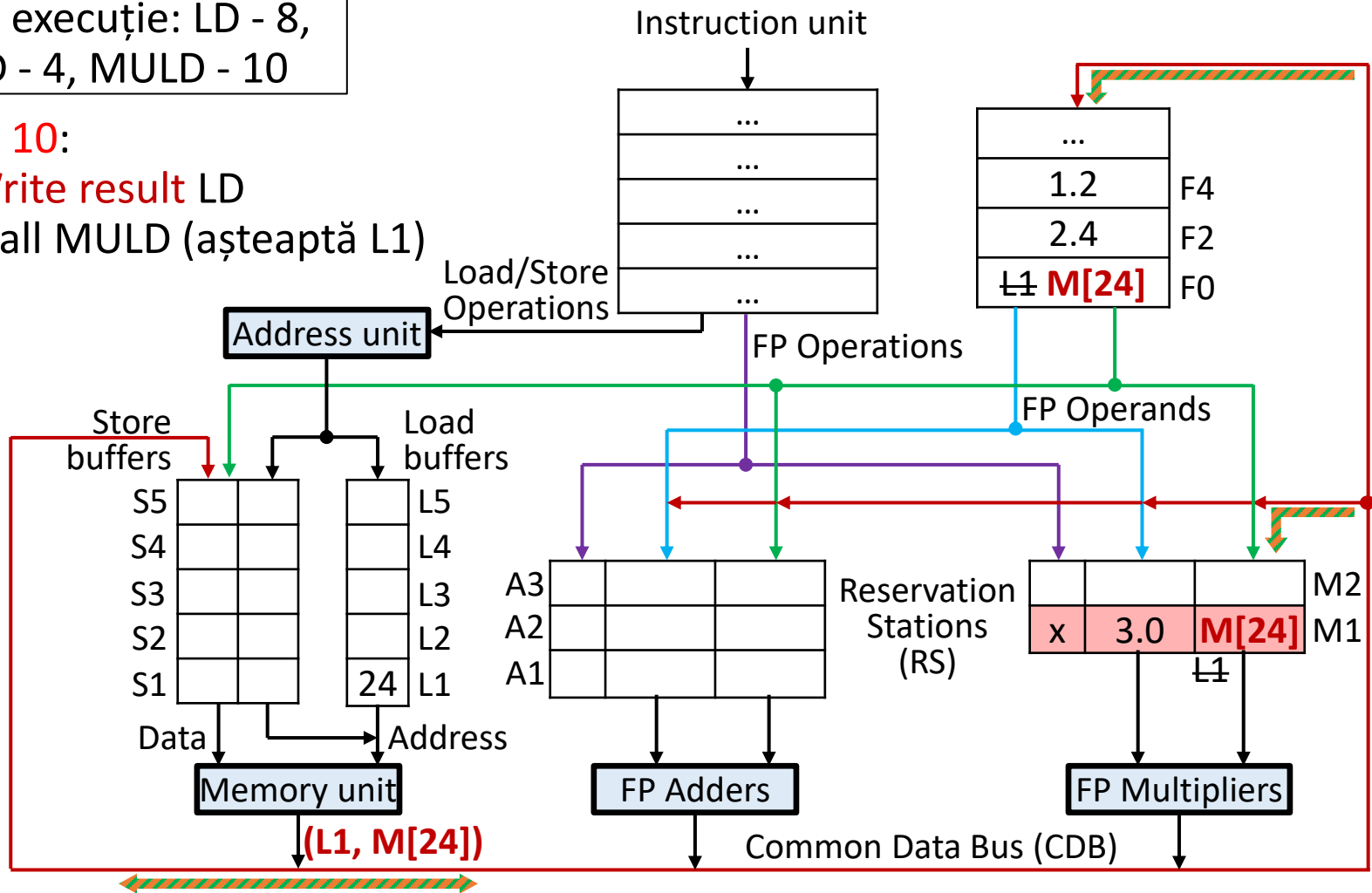
- S-a finalizat adunarea ADDD și se difuzează **2.4** pe CDB cu ID-ul **A1** (*write result*) => peste tot se înlocuiește A1 cu valoarea 2.4.
- Se eliberează stația A1.

Exemplu (cont)

Timpi execuție: LD - 8, ADDD - 4, MULD - 10
--

Ciclu 10:

- Write result LD
- Stall MULD (așteaptă L1)



Evenimente:

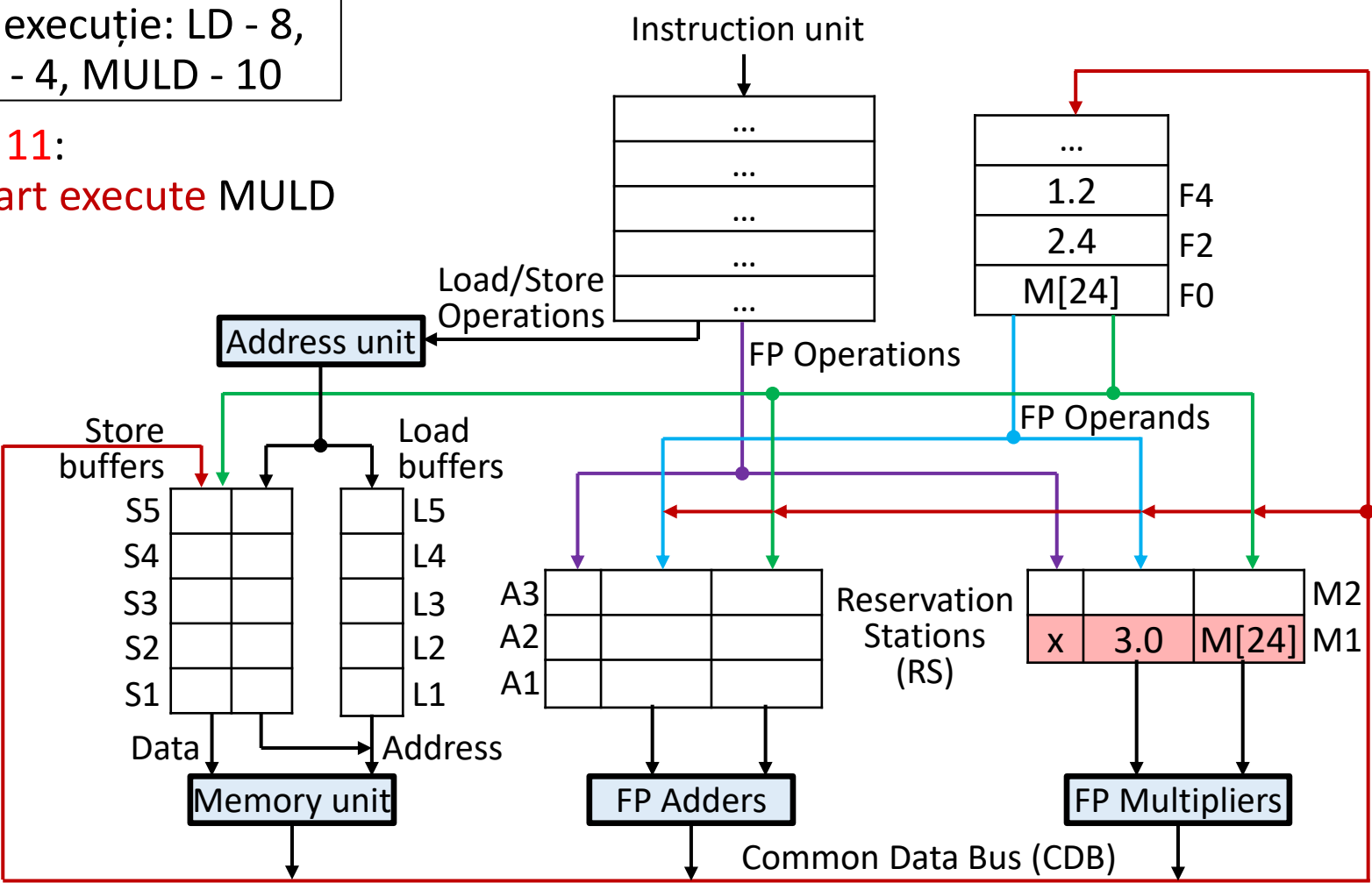
- S-a finalizat citirea din memorie și se difuzează **M[24]** pe CDB cu ID-ul **L1** (*write result*) => peste tot se înlocuiește L1 cu valoarea M[24].
- Se eliberează buffer-ul L1.

Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 11:

- Start execute MULD



Evenimente:

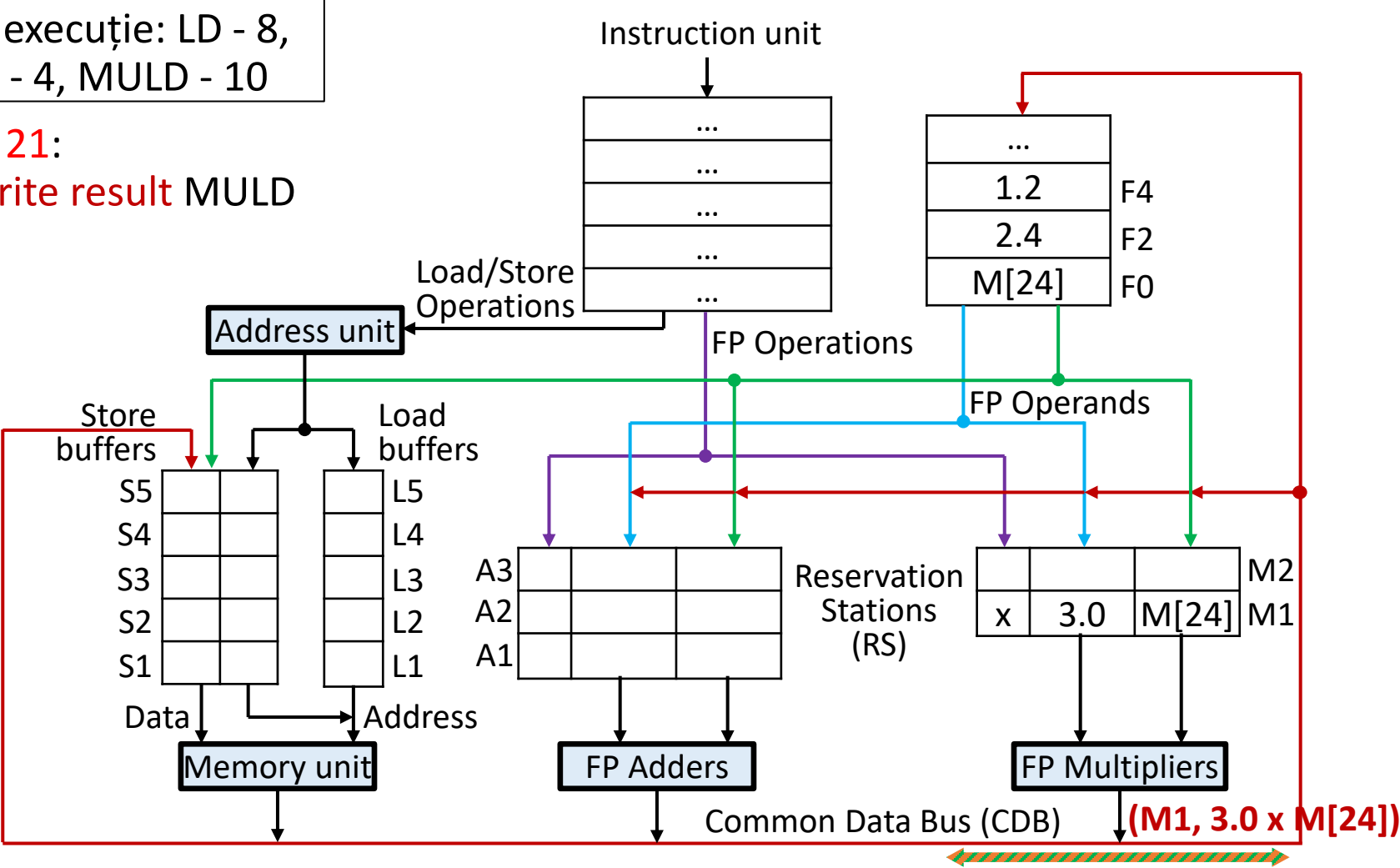
- Stația M1 are toți operanzii, începe execuția MULD.

Exemplu (cont)

Timpi execuție: LD - 8,
ADDD - 4, MULD - 10

Ciclu 21:

- Write result MULD

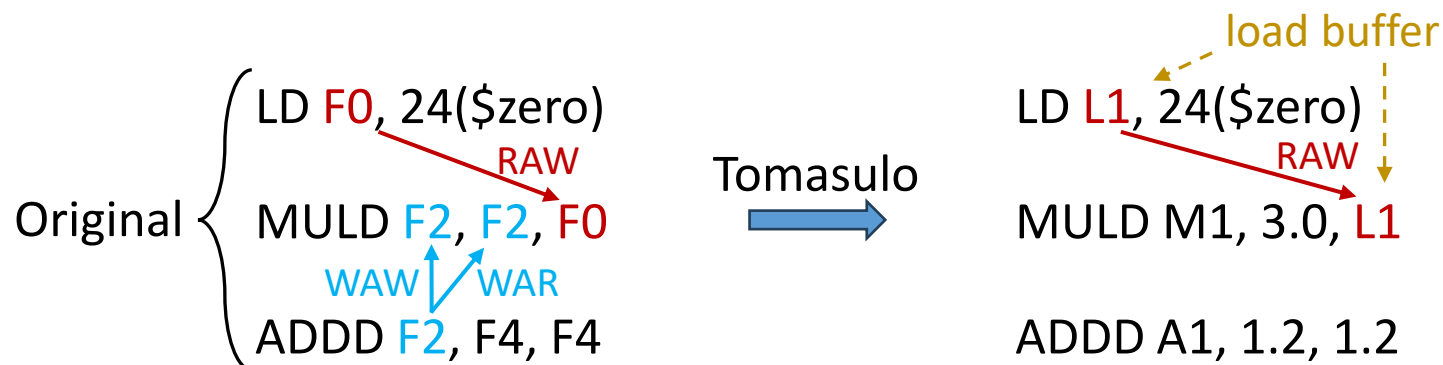


Evenimente:

- S-a finalizat înmulțirea MULD și se difuzează $3.0 \times M[24]$ pe CDB cu ID-ul **M1** (*write result*) => peste tot se înlocuiește M1 cu valoarea $3.0 \times M[24]$.
- Se eliberează stația M1.
- **Obs:** M1 nu mai scrie niciun registru.

Exemplu (cont)

Efectul redenumirii registrelor cu planificare dinamică



Redenumirea registrelor – avantaje

Redenumirea registrelor + așteptarea datelor difuzate pe CDB elimină hazardurile RAW, WAW și WAR.

Planificare dinamică – justificare

- De ce planificare? Pentru că lansarea instrucțiunilor noi, în timp ce altele sunt blocate, este aproximativ echivalentă cu planificarea software (instruction scheduling), care intercalează instrucțiuni independente, între cele dependente, pentru a le lansa în execuție și a evita eventuale **Stalls**.
- De ce dinamică? Pentru că se realizează pe loc (*on-the-fly*) în timpul execuției.

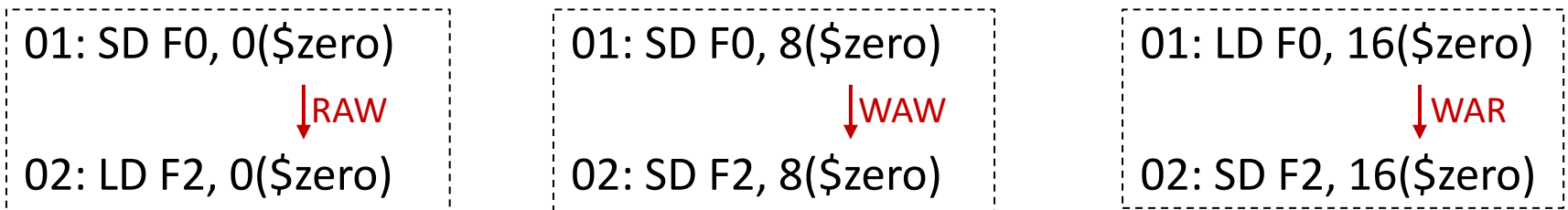
9.4 Planificarea dinamică: Tomasulo

9.4.6 Hazard de date la memorie

- Există situații excepționale când un load/store poate dura mai multe cicluri de ceas (chiar sute de mii!).
- Hazardul poate apărea între instrucțiuni load/store care accesează aceeași adresă de memorie.

Exemple

Presupunem că apar întârzieri și instrucțiunea 02 se încheie înaintea instrucțiunii 01.



Obs: Nu apare hazard între load-uri succesive => se pot executa în orice ordine.

Soluția Tomasulo – **Dynamic memory disambiguation**

- Adresele se calculează în *Address Unit*, **în ordinea din program** (secvențial).
- a) Unui load nu i se alocă Load buffer (este **stalled**) până când adresa sa diferă de cele existente deja în buffer-ele store (**elimină posibilitatea de hazard RAW**).
- b) Unui store nu i se alocă Store buffer (este **stalled**) până când adresa sa diferă de cele existente deja în buffer-ele Load (**elimină WAR**) și Store (**elimină WAW**).
- Odată ajunse în buffer, instrucțiunile load/store se pot executa în orice ordine.

9.4 Planificarea dinamică: Tomasulo

9.4.7 Despachetarea buclelor (loop unrolling)

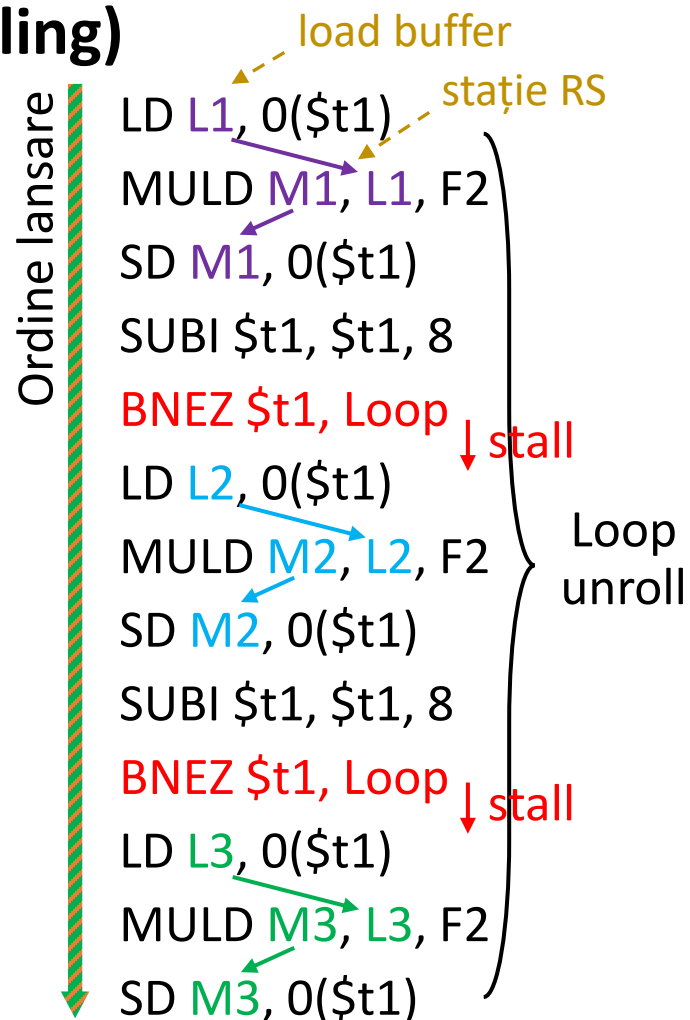
Exemplu

C++:

```
for (i=1000; i>0; i--)  
  x[i] += S;
```

Loop: LD **F0**, 0(\$t1)
MULD **F4**, **F0**, F2
SD **F4**, 0(\$t1)
SUBI \$t1, \$t1, 8
BNEZ \$t1, Loop

Tomasulo
redenumirea
registrelor



Notă: Presupunem că sunt mai multe stații RS disponibile și instrucțiunile LD, MULD durează un număr mare de cicluri față de restul instrucțiunilor, deci eliberează stațiile după mai multe iterații “despachetate”.

Efectul produs: Datorită planificării dinamice, se pot executa în paralel instrucțiuni de la iterații diferite, angrenând eficient cât mai multe unități funcționale.

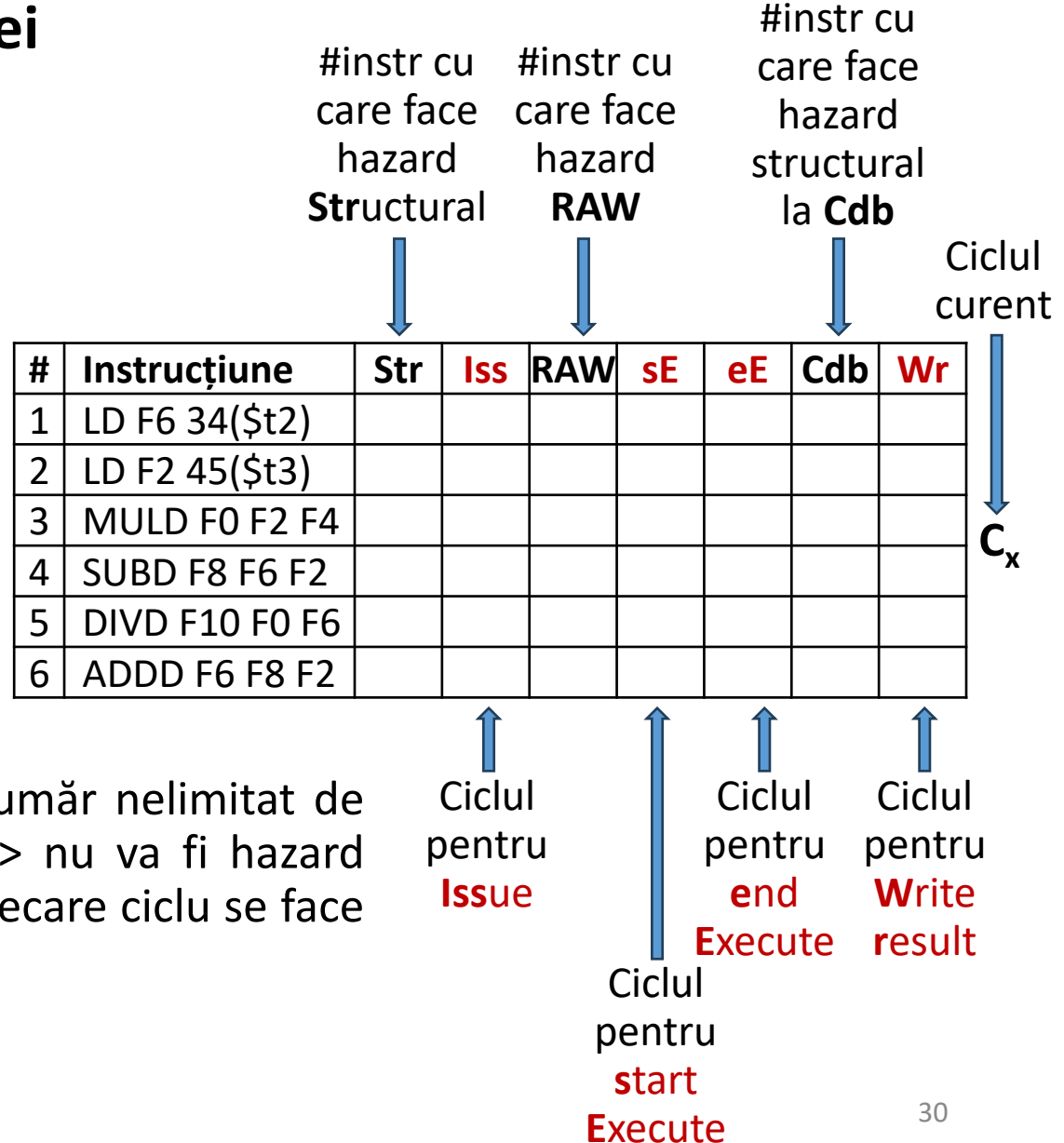
Concluzie: Tomasulo realizează implicit despachetarea buclelor, cu unele limite impuse de timpul necesar deciziei de branch: cu cât se ia decizia mai devreme, va începe mai devreme execuția instrucțiunilor din noua iterație.

9.4 Planificarea dinamică: Tomasulo

9.4.8 Planificarea execuției

Exemplu de secvență de cod

Timpi de execuție:
LD - 2, ADDD/SUBD - 2,
MULD - 10, DIVD - 40



Notă: Presupunem că sunt un număr nelimitat de stații RS și buffere Load/Store => nu va fi hazard structural în prima coloană și la fiecare ciclu se face **Issue** la câte o nouă instrucțiune.

9.4.8 Planificarea execuției

Exemplu (cont)

Timpi de execuție:
LD - 2, ADDD/SUBD - 2, MULD - 10, DIVD - 40

Explicații

- După **Issue** instrucțiunile încep execuția dacă nu întâmpină hazard RAW, adică stațiile/buffer-ele lor nu trebuie să aștepte niciun rezultat difuzat pe CDB.
- Dacă 2 instrucțiuni (stații/buffere) termină execuția simultan și vor să difuzeze deodată pe CDB (hazard structural), una din ele va aștepta un ciclu. Este permisă numai o singură difuzare CDB / ciclu.

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1					
2	LD F2 45+\$t3							
3	MULD F0 F2 F4							
4	SUBD F8 F6 F2							
5	DIVD F10 F0 F6							
6	ADDD F6 F8 F2							

C₁

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	<u>3</u>		
2	LD F2 45+\$t3		2					
3	MULD F0 F2 F4							
4	SUBD F8 F6 F2							
5	DIVD F10 F0 F6							
6	ADDD F6 F8 F2							

C₂

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		
2	LD F2 45+\$t3		2		3	<u>4</u>		
3	MULD F0 F2 F4		3					
4	SUBD F8 F6 F2							
5	DIVD F10 F0 F6							
6	ADDD F6 F8 F2							

C₃



Marcaj
când se
termină
execuția

9.4.8 Planificarea execuției

Exemplu (cont)

Timpi de execuție:
LD - 2, ADDD/SUBD - 2, MULD - 10, DIVD - 40

Str	Iss	RAW	sE	eE	Cdb	Wr
	1		2	3		
	2		3	4		
	3					

- Explicații** **C₃**
- La ciclul C₄, MULD nu începe execuția fiindcă se așteaptă difuzarea pe CDB a valorii citite din memorie de #2:LD (hazard RAW) .
 - La C₅ nici SUBD nu începe execuția din motive similare (hazard RAW).
 - La C₅ se difuzează pe CDB valoarea așteptată de MULD și SUBD. Începe execuția lor la C₆.
 - La C₆ DIVD nu începe execuția, ci așteaptă difuzarea pe CDB a rezultatului MULD (RAW).

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		
3	MULD F0 F2 F4		3	2				
4	SUBD F8 F6 F2		4					
5	DIVD F10 F0 F6							
6	ADDD F6 F8 F2							

C₄

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2				
4	SUBD F8 F6 F2		4	2				
5	DIVD F10 F0 F6		5					
6	ADDD F6 F8 F2							

C₅

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	15		
4	SUBD F8 F6 F2		4	2	6	7		
5	DIVD F10 F0 F6		5	3				
6	ADDD F6 F8 F2		6					

C₆

9.4.8 Planificarea execuției

Exemplu (cont)

Timpi de execuție:
LD - 2, ADDD/SUBD - 2, MULD - 10, DIVD - 40

Str	Iss	RAW	sE	eE	Cdb	Wr
	1		2	3		4
	2		3	4		5
	3	2	6	<u>15</u>		
	4	2	6	<u>7</u>		
	5	3				
	6					

C₆

Explicații

- La C₇, ADDD nu începe execuția fiindcă așteaptă difuzarea pe CDB a rezultatului SUBD (RAW) .
- La C₈ se difuzează pe CDB valoarea așteptată de ADDD și începe execuția sa la C₉.
- La C₁₁ se finalizează ADDD.

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	<u>15</u>		
4	SUBD F8 F6 F2		4	2	6	7		
5	DIVD F10 F0 F6		5	3				
6	ADDD F6 F8 F2		6	4				

C₇

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	<u>15</u>		
4	SUBD F8 F6 F2		4	2	6	7		8
5	DIVD F10 F0 F6		5	3				
6	ADDD F6 F8 F2		6	4				

C₈

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	<u>15</u>		
4	SUBD F8 F6 F2		4	2	6	7		8
5	DIVD F10 F0 F6		5	3				
6	ADDD F6 F8 F2		6	4	9	10		11

C₁₁

9.4.8 Planificarea execuției

Exemplu (cont)

Timpi de execuție:
LD - 2, ADDD/SUBD - 2, MULD - 10, DIVD - 40

Str	Iss	RAW	sE	eE	Cdb	Wr
	1		2	3		4
	2		3	4		5
	3	2	6	15		
	4	2	6	7		8
	5	3				
	6	4	9	10		11

C₁₁

Explicații

- La C₁₆ se difuzează pe CDB valoarea așteptată de DIVD și începe execuția sa la C₁₇.
- La C₅₇ se finalizează DIVD.

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	15		16
4	SUBD F8 F6 F2		4	2	6	7		8
5	DIVD F10 F0 F6		5	3				
6	ADDD F6 F8 F2		6	4	9	10		11

C₁₆

#	Instrucțiune	Str	Iss	RAW	sE	eE	Cdb	Wr
1	LD F6 34+\$t2		1		2	3		4
2	LD F2 45+\$t3		2		3	4		5
3	MULD F0 F2 F4		3	2	6	15		16
4	SUBD F8 F6 F2		4	2	6	7		8
5	DIVD F10 F0 F6		5	3	17	56		57
6	ADDD F6 F8 F2		6	4	9	10		11

C₅₇

Obs: Nu a apărut niciun hazard structural la CDB pe parcurs.

9.4 Planificarea dinamică: Tomasulo

9.4.9 Concluzii

Avantaje:

- **Planificare dinamică** a execuției: permite execuția instrucțiunilor, chiar dacă cele anterioare sunt blocate.
- Tomasulo = **in-order issue, out-of-order execute and completion**.
- **Rezolvarea hazardurilor WAW** prin mecanismul de redenumire a registrelor; **rezolvarea hazardurilor RAW și WAR** prin redenumire și așteptarea datelor pe CDB.
- Despachetarea buclelor (**loop unroll**).
- Eliminarea hazardurilor la memorie (**dynamic memory disambiguation**).

Dezavantaje:

- Limitări datorate **hazardului structural pe CDB** – se poate rezolva cu CDB multiple, dar crește complexitatea.
- Constrângerile pentru eliminarea hazardurilor la memorie impun **limite mai severe execuției instrucțiunilor de load/store** care accesează adrese similare.
- Blocarea execuției instrucțiunilor până la **finalizarea branch-urilor anterioare**, pentru a elimina hazardurile de control.

Referințe

- [1] J. L. Hennessy, D. A. Patterson, “Computer Architecture: A Quantitative Approach”, 5th edition, Cap. 3.2, 3.4, 3.5, Anexa C5, Ed. Morgan–Kaufmann, 2011.
- [2] D. A. Patterson, “Lecture 3: Introduction to Advanced Pipelining”, pg. 4-23, Computer Science 252, Spring 1998. Online (material suport cu exemple de execuție): <http://www.cs.berkeley.edu/~pattrsn/252S98/Lec03-ilp.pdf>
- [3] D. A. Patterson, “Lecture 4: Tomasulo Algorithm and Dynamic Branch Prediction”, pg. 9-62, Computer Science 252, Spring 1998. Online (material suport cu exemple de execuție): <http://www.cs.berkeley.edu/~pattrsn/252S98/Lec04-tomosulo.pdf>